The Soft Continuous Equal-Area Constraint (A2)

Thirty-two times faster, and twice fatally wrong: a measured rejection at N=100

August 2026

Abstract

Phase 1 spends 93.3% of its wall time in an iterative alternating projection whose only job is to enforce equal *continuous* cell mass — a constraint whose delivered meaning, an equal-area winner-take-all partition, is now guaranteed independently and exactly by the balanced readout at extraction time. This report measures what happens when that constraint is weakened to a soft quadratic penalty, leaving sum-to-one plus box: a closed-form, non-iterative per-vertex simplex projection. At N=100 the change is **32**× faster and kills no cells — the pre-registered falsifier does not fire — but it is **rejected**, on two *independent* defects that were not visible in the headline numbers. (1) The penalty's gradient is a *uniform per-cell field*, so a cell short on area is boosted everywhere at once, including at a distant foothold: fragmented cells accumulate $0 \to 4$ exactly as imbalanced cells fall $79 \to 0$. The flow buys area balance with disconnected territory. (2) The Euclidean simplex projection maps a whole small-step neighbourhood of a crisp iterate back onto the same simplex vertex, so $E_{\text{trial}} = E$ and the Armijo test can never be satisfied: *every* level collapses to a step of 9.095×10^{-13} and freezes, and the energy-plateau trigger then fires on the frozen energy and reports convergence. Most of the 32× is therefore not a speedup at all but a dead optimizer. Defect (1) was predicted in advance by the locality criterion of `docs/reference/winner_take_all_partition_gap.md` §9b, which now has two independent confirmations.

Status — measured; verdict REJECTED

All numbers below are measured at N=100 against a matched control that was already a validated deliverable. Both defects are measured, not inferred. The approach is closed: the code remains on the branch behind a default-off flag (`soft_area_constraint`) as the record, and should not be enabled. **One narrower variant survives**: exact treatment at level 0, soft from level 1. It removes the fragmentation entirely, passes all three gates at 6.83×, and — *via the balanced readout, which it requires* — reaches a Phase 2 perimeter +0.246% off the control. See §6.

Provenance

Date	2026-08-10 / 2026-08-11; hybrid variant 2026-08-12
Surface	torus $T(R{=}1.0,\ r{=}0.6)$, $N{=}100$, $\lambda{=}5.1$, seed 84172851, seeded init, 5 levels ($100{\times}96 \to 348{\times}328$, $V{=}114{,}144$)
Control	`results/run_20260709_081548_surftorus_npart100_..._lam5.1_seed84172851` — the validated $N{=}100$ deliverable: 0 dead / 0 weak / 0 imbalanced / 0 fragmented, worst cell 0.78%, exported Phase 2 perimeter 185.2546. Its connectivity gate postdates the run and was computed retroactively for this report.
A2 arm	config `parameters/torus_100part_coarse_seeded_softarea.yaml` — identical in every relaxation knob except `soft_area_constraint: true`, `soft_area_mu: 245.9`.
Probe	the same config with `refinement_levels: 1`, `max_iter: 10000`, `enable_refinement_triggers: false`, `h5_save_stride: 250` — level 0 run past its trigger point to separate "stopped early" from "settled".
Scripts	`docs/experiments/05-soft-area-constraint/make_figures.py` (figures); `testing/calibrate_soft_area_mu.py` (μ); `testing/check_fragmentation.py` (gates); `testing/test_soft_area_constraint.py` (correctness).
Versions	Python 3.12.13, numpy 2.4.4, scipy 1.17.1, matplotlib 3.10.8, h5py 3.16.0; macOS 15.3.1 arm64 (Mac mini), `OMP_NUM_THREADS=2`.
Anchors	control Phase 1 wall 48,131.8 s (sum of `level_wall_s`); A2 Phase 1 wall 1,496.6 s; A2 level-0 energy frozen at 119.5651878371 from iteration 2,750 to 9,999; every A2 level's final step = 9.0949×10^{-13}.

Contents

1 The question

The Phase 1 timing profile is lopsided. At N=300, λ=11.5 (`docs/math/04-phase1-timing-profile/`), the alternating projection is 93.26% of wall time — 21.96 h across 6,762 major iterations at a mean of 46.3 projection inner iterations per call — while energy, gradient and backtracking together are 6.49%. A later five-level N=300 run put the projection share at 94.80% of 57.3 h. The prize is real and it is essentially the whole of Phase 1.

That projection enforces equal *continuous* cell mass, $\sum_i v_i u_{i,k} = \bar{A}$. Its purpose is to deliver an equal-area *winner-take-all* partition. Since the balanced readout (`src/partition/balanced_readout.py`) now delivers exactly that, by construction, to one-vertex granularity, at any N, at extraction time and in seconds, the constraint's job is done twice. So: weaken it to a soft penalty and keep only sum-to-one plus box, which is a *closed-form* per-vertex simplex projection with no inner loop at all.

The pre-registered falsifier. Runt (area-imbalanced) cells were *expected* under a soft constraint and are not disqualifying — the readout repairs them. The hypothesised danger was different: the hard mass constraint may be the only thing keeping a cell *alive* early in the flow. Any dead or weak cell would fail the approach regardless of speedup. That falsifier is stated in the config header and was fixed before the run.

2 Method

2.1 The change

Equal area moves from the feasible set into the objective:

$$P_{\text{area}} = \frac{\mu}{2}\sum_k r_k^2, \qquad r_k = \frac{v^\top u_k - \bar{A}}{\bar{A}}, \qquad \frac{\partial P_{\text{area}}}{\partial u_{i,k}} = \frac{\mu}{\bar{A}^2}\, v_i\,(v^\top u_k - \bar{A}),$$

and the per-trial projection in the line search becomes the Euclidean projection of each row onto the unit simplex (sort-and-threshold), replacing the alternating projection. Two implementation choices matter for reading the results:

- The **entry projection** (once per level, on the initial iterate) stays exact in both arms, so each level starts from the identical feasible point and the comparison isolates the flow.
- `constraint_fun` **drops the area block** when the flag is on, so `FEAS` keeps describing the constraints actually enforced. Otherwise `FEAS` would sit permanently above `refine_constraint_tol` and the refinement trigger would silently mean something different in the two arms, invalidating the comparison.

2.2 Calibration of μ

μ is calibrated, not chosen: `testing/calibrate_soft_area_mu.py` picks the μ at which the penalty force reaches parity with the base energy's $\|g\|_\infty$ at a 5% relative area deviation — the discrete-area gate threshold — evaluated at the level-0 seeded initial condition. This gives $\mu = 245.9$ at N=100 (and 895.3 at N=30; μ scales with vertices-per-cell and is not N-invariant). Stiffness was checked rather than assumed: the penalty adds ≈ 302 to the Hessian's top eigenvalue, permitting steps to 6.6×10^{-3}, while this config's level-0 accepted steps already fall to 4.9×10^{-4} — so the penalty is not the step-limiting term. *This matters for §5: the line-search collapse reported there is not a stiffness artifact of μ.*

2.3 Correctness gates before any run

`testing/test_soft_area_constraint.py`: the simplex projection against the KKT conditions (stationarity residual 4.4×10^{-16}, feasibility 2.7×10^{-15}, box-exact, idempotent); the penalty gradient against central differences (3.3×10^{-8}); the wiring against an independently written closed form; and flag-off bit-identity of energy, gradient and constraint vector.

3 Result 1 — the headline, which is misleading

	control	A2
Phase 1 wall	48,132 s (13.37 h)	**1,497** s (0.42 h)
speedup	—	**32.2**×
mean projection inner iterations	34.0	1.03
dead / weak cells	0 / 0	**0/0**
imbalanced (worst)	0 (0.78%)	0 (1.47%)
continuous area drift	∼0 (hard)	2.06%
sum-to-one feasibility	1.3×10^{-10}	$\mathbf{6.7 \times 10^{-16}}$
fragmented cells	**0**	**3** (worst stray 46.3%)

The falsifier did not fire: no cell died. The soft constraint also held its own target far better than required (continuous drift 2.06%, discrete imbalance 1.47%), and sum-to-one came out six orders of magnitude *tighter* than the control, because the closed-form projection is exact where the alternating one merely reaches its stopping tolerance.

It fails on a third axis instead. Three cells are split. The balanced readout does repair them — 0 imbalanced, 0 fragmented — but at 13,540 moves against the control's 51, 213 blocked against 0, hitting the 50-sweep cap, and costing +22.0% label-boundary length against the control's +0.13%. Phase 2 inherits that geometry and **aborts**: IPOPT reports `EXIT: Restoration Failed!` at topology iteration 3 of 20, having exhausted `max_opt_iter` on the two before it. The best perimeter reached, 197.40, is where the solver died, not a converged value; the control converges to 185.2546 in 20 iterations with no solver failure.

4 Result 2 — defect (1): balance is bought with disconnection

Running level 0 past its trigger point, with triggers disabled, separates "stopped early" from "settled" and shows where the splits come from.

iteration	250	500	1500	2500	2750	→9999
fragmented cells	0	1	2	3	**4**	**4**
imbalanced cells	79	63	22	1	**0**	**0**

Fragmentation appears at iteration 500 and accumulates *monotonically as imbalance is driven out*, reaching 4 cells by iteration 2,750 and staying there — worse than the 3 the production arm ended with, so running longer makes it worse, not better. The mechanism is in the gradient: the penalty contributes $(\mu/\bar{A}^2)\, v_i\, (v^\top u_k - \bar{A})$, a field that is *uniform across cell k* up to the vertex weight v_i. A cell short on area is therefore boosted everywhere at once — including wherever it holds a small distant foothold, which grows into an island.

This was predicted. §9b of `docs/reference/winner_take_all_partition_gap.md` was written before A2 existed, to explain why the (rejected) discrete-area trim manufactured islands, and it states the test: *through what operator does the correction reach the field, and is that operator*

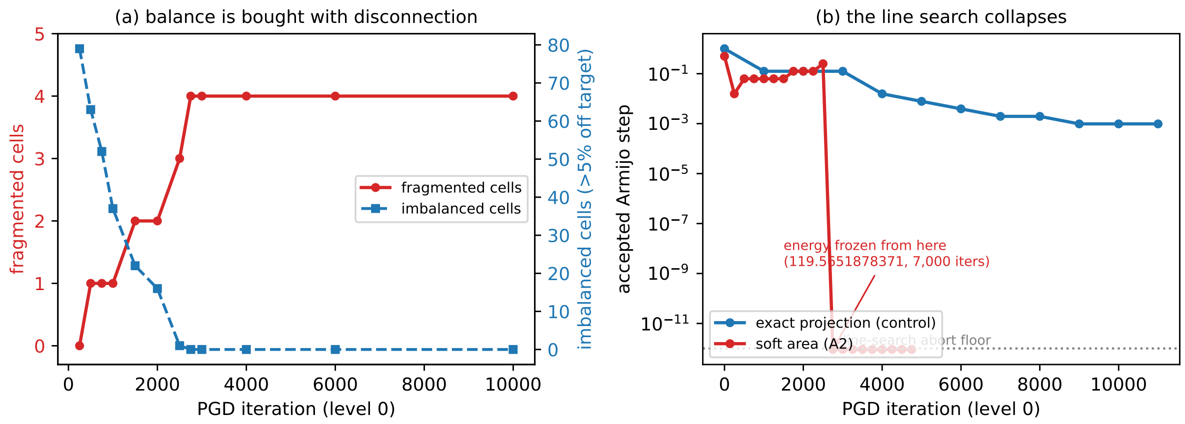

Figure 1: The two independent defects. **(a)** At level 0 the fragmented-cell count rises 0 → 4 exactly as the imbalanced-cell count falls 79 → 0: the flow buys area balance with disconnected territory, and then freezes there. **(b)** The accepted Armijo step collapses from 2.5×10^{-1} to 9.095×10^{-13} between iterations 2,500 and 2,750 and the energy freezes to the digit, while the control's step decays gracefully across 30,000 iterations and never collapses.

local? The trim reached the field through the projection, a nonlocal operator. A2 reaches it through a uniform per-cell push — nonlocal in exactly the same sense. Two mechanisms that look nothing alike, one criterion, one symptom. See §8.

5 Result 3 — defect (2): the line search collapses

The second defect is independent of the first and is the more serious one for interpreting the headline.

level	iterations run	step collapses at	final step
A2 level 0	2,575	2,548	9.095×10^{-13}
A2 level 1	282	257	9.095×10^{-13}
A2 level 2	211	182	9.095×10^{-13}
A2 level 3	184	154	9.095×10^{-13}
A2 level 4	257	232	9.095×10^{-13}
control level 0	30,000	**never**	healthy

Every A2 level dies some 25–30 iterations before its "convergence" trigger fires. In the probe, the energy is bit-identical at 119.5651878371 from iteration 2,750 through 9,999 — seven thousand iterations in which nothing is accepted — with a mean of 30.3 backtracks per iteration and 11.3% of wall time spent backtracking. The control never collapses at all.

Why. Once the field is crisp, the iterate sits at a vertex of the box-truncated simplex. The Euclidean projection maps an entire small-step neighbourhood of that point back onto the same vertex, so $x_{\text{trial}} = P(x - sg) = x$ exactly and $E_{\text{trial}} = E$, while the Armijo test demands $E_{\text{trial}} \leq E - c\,s\,\|g\|^2$ with $c\,s\,\|g\|^2 > 0$. No step size can satisfy it. The search backtracks ~ 40 times to below the 10^{-12} abort floor and repeats forever. The alternating projection's multiplicative renormalisation has no such fixed point: it keeps entries strictly positive, so small steps produce small but nonzero motion.

Consequence for the headline. A2's 32× decomposes into 3.1–6.4× per-iteration cost and 4.7–11.7× fewer iterations. The second factor is *not a saving*: it is the optimizer dying at every level and the energy-plateau trigger, which cannot distinguish a frozen energy from a converged one, reporting the failure as convergence.

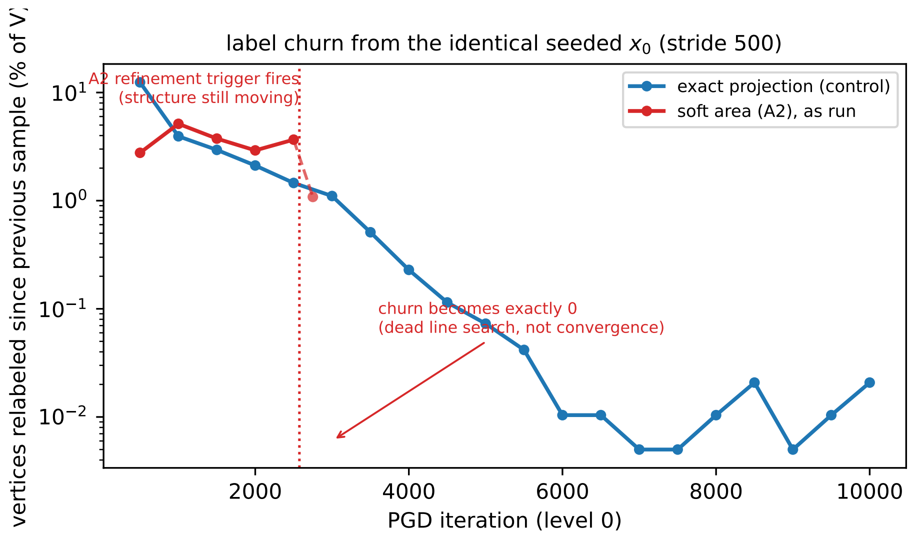

Figure 2: Winner-take-all label churn at level 0 from the identical seeded x_0. The control decays monotonically; the soft-area arm does not, and was still relabelling 3.7% of vertices per 500 iterations when its trigger fired at iteration 2,575. Past that point the churn does not become small — it becomes *exactly* zero, for 7,000 iterations, which is the signature of a dead line search rather than a settled structure.

A diagnostic worth stealing. Exactly-zero churn is qualitatively different from small churn, and it is the cheapest tell that an optimizer has stopped rather than converged. Convergence gets *cheaper* per iteration; this got 13× more expensive per iteration at the moment it stopped moving. A guard now warns when no step is accepted for 50 consecutive iterations (`src/optimization/pgd_optimizer.py`), so this failure cannot again be mistaken for a speedup.

6 The hybrid variant — also closed

One variant is not refuted by the above: run level 0 on the *exact* treatment and levels 1–4 on the soft one (`soft_area_from_level: 1`). Fragmentation is born at level 0 (§4), and A2's finer levels never added splits beyond what level 0 handed them, so a clean level-0 structure may survive. Both halves of the treatment are gated together deliberately — penalty *and* projection — since they are separate defects and swapping only one would carry the other into the level it was built to protect.

Measured (2026-08-12). The prediction held on the mechanism and the result is more interesting than expected.

	control	A2	hybrid
Phase 1 wall	48,132 s	1,497 s	**7,044** s (6.83×)
level-0 line search	healthy	collapses @2,548	**never collapses**
levels 1–4 collapse at	—	257/182/154/232	425/130/150/228
dead / weak	0 / 0	0 / 0	0 / 0
imbalanced (worst)	0 (0.78%)	0 (1.47%)	0 (1.21%)
fragmented	**0**	**3**	**0**
initial perimeter	214.34	216.79	**213.23**
Phase 2	185.2546 (20 iters)	abort @3	**abort @1** (187.34)

Two findings. First, **it confirms §4 directly**: making level 0 exact removes the fragmentation entirely (0 cells, no raw components at all), so the splits really are born at level 0 and inherited, and the hybrid passes all three validity gates at 6.83× — better than the ≈3.8× predicted, because the collapsing fine levels are *cheap* as well as useless. Second, levels 1–4 collapsed exactly as predicted (after 130–425 iterations), so they contribute almost no refinement, and **Phase 2 still aborts** — at iteration 1, `Restoration Failed!`.

That last point is not explained by anything measured here and is stated as an open residual rather than a conclusion: the partition handed to Phase 2 is structurally near-identical to the control's (14,623 variable points and 200 triple points against 14,641 and 200), passes all three gates, and has a *better* extracted perimeter (213.23 vs 214.34) — yet IPOPT cannot restore feasibility from it. Whatever Phase 2 needs from the fine levels, the three Phase 1 validity gates do not measure it.

The abort was the path, not the partition (2026-08-12). The Phase 2 run above was deliberately made on the *raw* solution, to match how the control's 185.2546 was produced. That is the right like-for-like comparison and the wrong production path: every successful high-N Phase 2 on this project goes through the balanced readout first. Re-run through it:

readout strain	moves	blocked	boundary	sweeps
control	51	0	+0.13%	12
hybrid	**426**	**0**	**+0.58%**	**31**
N=300 five-level (known good)	3,305	0	+1.85%	76
A2, raw (broken)	13,540	213	+22.0%	cap hit

The hybrid's strain is squarely in the healthy band — zero blocked moves, sub-1% boundary cost — so its geometry was never the problem. Worst-cell deviation falls 1.21% → 0.20%, and **Phase 2 then completes 20 topology iterations with zero solver failures**, best perimeter 185.7108 against the control's 185.2546: +0.246%.

What actually predicted the abort. Phase 2's iteration-0 constraint violation is the worst-cell area deviation times the target area $\bar{A}$, and that relation now holds on three runs to within 3%:

	worst cell	predicted	Phase 2 reported
control (raw)	0.78%	1.85×10^{-3}	1.85×10^{-3} — converges
hybrid (raw)	1.21%	2.87×10^{-3}	2.87×10^{-3} — **aborts**
hybrid (readout)	0.20%	4.74×10^{-4}	4.59×10^{-4} — converges

A 1.55× larger starting violation was the whole difference between a 20-iteration descent and a restoration failure in 8 seconds. This is the cheapest single predictor of whether Phase 2 will descend, and it is available from the Phase 1 solution before Phase 2 is started.

Verdict — viable, with a dependency. The hybrid gives **6.83**× on Phase 1 with all three gates passing and a Phase 2 perimeter +0.246% off the control. That is a real quality cost, not noise — ten times the −0.024% spread the structure trigger showed — bought for a 6.83× reduction in the dominant cost of the pipeline.

The honest framing is that the hybrid *requires* the readout, not that Phase 2 was fine all along. On the raw path it fails outright. This promotes the balanced readout from an optional repair to a **mandatory stage of the fast path**, and any future use of the hybrid inherits that dependency.

Two cautions on the arithmetic. The 6.83× **already contains** the structure trigger (level 0 is 6,099 s of the 7,044 s because the trigger stops it at 6,500 iterations); the two do *not* stack. Without the trigger the hybrid is 2.34×, and the trigger contributes 13,484 s of the 41,088 s saved. And the +0.246% may not hold at higher N: the hybrid is fast partly because its fine levels collapse and do no refinement, so the perimeter cost of skipping that refinement should be re-measured at N=300 before the speedup is relied on.

7 Third attempt — the adaptive switch, also rejected

`soft_area_from_level` has to be read off a *finished* run, which makes it useless at the problem size that matters. The replacement decides during the run: each level starts exact and switches to soft once that level's own label churn falls below 10^{-5} flips per iteration per vertex over 500 iterations, with μ calibrated at the switch from the iterate in hand. Fixed rule, data-driven switch point — the same contract as the structure trigger, and the reason it could in principle be validated at one size and used at another.

The calibration worked. Replaying the rule offline on the control's churn traces predicted a level-2 switch at ≈4,250; the live run fired at **4,499**, before an hour of compute was spent. That answers the objection that in-experiment calibration is worthless: it need not be.

The mechanism did not. Graded at `checkpoint_level03.h5` against the control's final solution — same mesh (224 × 212), seed and λ, so like-for-like:

	control	adaptive
imbalanced (worst)	10 (36.15%)	**4 (15.36%)**
fragmented	**2** (274, 290)	**3** (274, 290, **299**)
cell 299	intact, a runt	**28 components**, 10 significant, stray 28.4%
cell 290	2 components, stray 4.1%	7 components, 5 significant, stray 23.7%
level-2 wall	21.55 h	15.08 h (1.43×)

The penalty did exactly what it was built to do — imbalance 10 → 4, worst deviation 36% → 15% — and bought it by shattering the worst runt into 28 pieces. **That is the wrong trade, and the asymmetry is the whole argument**: area imbalance is free to fix (the readout corrects it at any N in seconds, on three runs), while fragmentation is fixable only while the damage is small — 2 fragmented cells cost 3,305 moves / 0 blocked / +1.85%, whereas A2's damage level cost 213 blocked / +22% and Phase 2 refused to run. Fifteen significant stray pieces sits far nearer the second.

Third confirmation of the locality criterion. The offending line is `G += (mu / Abar**2) * np.outer(self.v, dev)` — a rank-one, uniform per-cell field, pushing a deficient cell up at *every* vertex where it has any presence, specks included. Three mechanisms have now failed §8's test: the territory-aware trim (nonlocal via the projection), A2 (nonlocal via this push), and the adaptive switch (the same push, later). The criterion has predicted every one.

An early warning that was visible live. Churn fell to 5.22×10^{-6}, the switch fired, and churn then *rose back* to 1.14×10^{-5} — above the very threshold that had declared the structure settled. That was not noise; it was cell 299 coming apart, and it was observable roughly 40 minutes before the level ended.

Two things that confound the verdict, and are left open. *(1) The restart is violent, and it is my implementation, not the mechanism.* On the switch the code reset the line-search warm start to `step0`, which is the most aggressive available choice rather than a neutral one. The transient:

iteration	E	$\lVert g \rVert$	step	FEAS
4,494–4,499 (settled, exact)	720.09	31.1	3.1×10^{-2}	9×10^{-9}
4,500 (first soft step)	**629.34**	397.0	**1.0**	—
4,501	620.60	332.7	3.9×10^{-3}	—
4,503+	611.31	291.6	2.0×10^{-3}	—

The first post-switch step is 32× the settled step, along a gradient 12.8× larger, and drops the energy by 90.75 in one iteration — and the search then backtracks to 16× *smaller* than the pre-switch step, which is the line search itself rejecting 1.0 as absurd. It was accepted only because releasing the hard area constraint made a large energy gain available for nothing. Continuous area deviation goes from $\sim 10^{-9}$ to **36.98%** in that single step, then recovers to 4.06% within 500 iterations and settles at 2–4%. So the excursion is transient; the structural damage it causes is not, because fragmentation is a stable fixed point (§4). **A damped first step would separate "the penalty is nonlocal" from "the restart is violent"** — the deficit that the nonlocal push then repairs destructively may be self-inflicted. One cheap test; not run.

(2) A churn-revert rule was available and absent. Reverting to exact (or aborting the level) when post-switch churn climbs back above the switch threshold would have caught this live, 40 minutes early. Also not built.

The collapse reproduces at N=300, and is not only a soft-phase defect. The soft phase stalled at iteration 5,724, 1,225 soft iterations after the switch, at the same 9.095×10^{-13} floor. But the guard also caught something that changes how §5 should be read: levels 0 and 1 — which never switched, being too short for the window, and therefore ran *entirely on the exact treatment* — stalled at iterations 31 and 41 and ended at 47 and 57 with the same floored step. Those are the control's exact iteration counts. So the N=300 control's own two coarsest levels also terminate on a dead line search; at 32 and 83 vertices per cell they are far below the resolution floor. Line-search collapse is therefore not unique to the Euclidean projection — it is what an under-resolved level does — though at N=100 the exact level 0 never collapsed across 30,000 iterations, so it is resolution-dependent rather than universal.

8 Conclusions

1. **The stated falsifier was the wrong one.** No cell died — the hard mass constraint is not what keeps cells alive. A2 failed on connectivity and on optimizer health, neither of which was the pre-registered risk. Pre-registering a falsifier remains right; expecting it to be the thing that fires is not.

2. **Connectivity is the fragile invariant of this problem.** Nothing in the energy, the constraints, or either replacement protects it. Two independent attempts to cheapen Phase 1 have now failed by manufacturing disconnected cells: the territory-aware term via nonlocal area transport through the projection, and A2 via a uniform per-cell push. Different mechanisms, identical symptom.

3. **The locality criterion is predictive, and should now be a hard gate rather than a screening test.** §9b stated it before A2 existed and called A2's failure mode correctly; it has since called the adaptive switch's too (§7). Three for three. No mechanism should get an arm until it can state which operator carries the correction and why that operator is local: *through what operator does the correction reach the field, and is that operator local?* A correction that reaches a cell uniformly, or through a global solve, will buy area with disconnected territory.

4. **Judge a pipeline on the pipeline.** The hybrid was first measured on the raw Phase 2 path, chosen for a defensible reason (matching how the control's number was produced), and it aborted. On the production path — the same solution through the balanced readout — it converges. A negative result from a path the project no longer uses is not a verdict on the approach.

5. **Do not price a speedup in iterations you did not have to run.** Most of the 32× was an optimizer dying early. A result that looks like a large speedup on a hard problem should be checked for a collapsed line search before it is believed.

Related documents

- `docs/reference/winner_take_all_partition_gap.md` — the standing explanation; §9b states the locality criterion this report confirms, and §4c records the screening test.
- `docs/experiments/04-territory-aware-highn-validation/` — the other mechanism that failed the same criterion.
- `docs/math/04-phase1-timing-profile/` — the projection cost that motivated the attempt.
- Code: `src/optimization/projection.py` (`project_rows_onto_simplex`), `src/optimization/pgd_optimizer` (penalty, gate, stall guard), `src/partition/balanced_readout.py` (what made the attempt thinkable).